

Reviewer Pack — Minimal Rank of Tropicalized Quandle Representations over Ma...

Entry b0db61a19b33 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 09:23:13 UTC

Minimal Rank of Tropicalized Quandle Representations over Max-CUT

Entry ID: b0db61a19b33 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 09:23:13 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Max-CUT

Statement:

{‘main’: ‘For a given max-CUT instance with n vertices, the minimal rank of its tropicalized quandle representation is at most $\Theta(\log n)$.’, ‘counterexample’: ‘There exists an instance where the minimal rank of the tropicalized quandle representation exceeds $\log n$.’}

Rationale (proposer’s reasoning):

{‘main’: ‘Quandle theory offers a structured way to encode combinatorial objects, and its application to Max-CUT may reveal new invariants that could help understand the complexity of the problem.’, ‘specific’: ‘Tropicalization allows for the analysis of quandles in a semi-algebraic setting, potentially leading to lower bounds for max-CUT.’}

Taxonomy category: SOS_DEGREE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4512f58980aa1f88

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The minimal rank of tropicalized quandle representations is at most $\Theta(\log n)$ for all Max-CUT instances with n vertices.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Quandle Theory' AND 'Max-CUT algorithm' AND 'tropicalization' - 'Minimal rank' AND 'quandle representation' AND 'complexity theory Max-CUT' - 'Tropicalized quandle representations' AND 'logarithmic rank bound' AND 'algorithmic complexity'

Top relevant hits considered: - [<http://arxiv.org/abs/1101.4803v2>] Lifting defects for nonstable K_0 -theory of exchange rings and C^* -algebras - [<http://arxiv.org/abs/1006.1381v1>] Categories parametrized by schemes and representation theory in complex rank - [<http://arxiv.org/abs/2105.11027v1>] Rank and Duality in Representation Theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        vertices = list(range(n))
        edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
        cut_edges = random.sample(edges, n - 1)
        return vertices, cut_edges

    def construct_quandle_representation(instance):
        vertices, _ = instance
        quandle_rep = {v: v for v in vertices}
        for u, v in generate_max_cut_instance(len(vertices))[1]:
            if quandle_rep[u] != quandle_rep[v]:
                quandle_rep[quandle_rep[v]] = quandle_rep[u]
        return quandle_rep

    def minimal_rank(quandle_rep):
        rank = 0
        visited = [False] * len(quandle_rep)
        for v in quandle_rep:
            if not visited[v]:
                stack = [v]
                while stack:
                    current = stack.pop()
```

```

        if not visited[current]:
            visited[current] = True
            for neighbor in quandle_rep:
                if quandle_rep[neighbor] == quandle_rep[current]:
                    stack.append(neighbor)
            rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
ranks = []

for n in n_values:
    instance = generate_max_cut_instance(n)
    quandle_rep = construct_quandle_representation(instance)
    rank = minimal_rank(quandle_rep)
    ranks.append(rank)

mean_rank = sum(ranks) / len(ranks)
conjecture_holds = all(rank <= math.log(n, 2) for n, rank in zip(n_values, ranks))
counterexample = "" if conjecture_holds else f"n={n_values[0]}, rank={ranks[0]}"

return {
    "metric_name": "Minimal Rank",
    "metric_value": mean_rank,
    "instances_tested": len(ranks),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction=1.0")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='n={n_values[0]}, rank={ranks[0]}' first_failing={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
d': 6, 'conjecture_holds': False, 'counterexample': 'n=5, rank=3'}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 10.5, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 12.166666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 10.666666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.166666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 10.333333333333334, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.333333333333334, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 10.666666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.166666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.666666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 10.5, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.166666666666666, 'instances_tested': 6, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Minimal Rank', 'metric_value': 11.5, 'instances_tested': 6, 'conjecture_holds': True}
RESULT: INCONCLUSIVE reason=insufficient_support_fraction support_fraction=0.0
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considered $n \leq 5$, which is too small to draw a definitive conclusion about the conjecture's validity for larger values of n . The metric does not scale trivially with n , but testing such a limited range is insufficient to confirm the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test provides a counterexample with $n=5$ where the minimal rank of the tropicalized quandle representation exceeds $\log(n)$. Specifically, the rank is 3. | next: Further investigation is needed to determine if there are instances for larger values of n that also violate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11944
2	propose	ollama_remote	glm4:latest	0	0	12956
3	propose	ollama_remote	glm4:latest	0	0	10440
4	preregistration	ollama_remote	glm4:latest	0	0	5201
5	novelty	ollama_remote	glm4:latest	0	0	5027
6	novelty	ollama_remote	glm4:latest	0	0	10502
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38549
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9757
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8982
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9473
11	critic	ollama_remote	glm4:latest	0	0	12218
12	judge	ollama_remote	glm4:latest	0	0	5721

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 140771 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in research/audit/b0db61a19b33.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b0db61a19b33.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b0db61a19b33.tar.gz (if generated)